

Lab session 3: Implementation of digital systems

– Real-time signal analysis: time sequence analysis and FFT

EMBEDDED SIGNAL PROCESSING SYSTEMS HT 2018

Instructions

- This lab session is intended for you to implement signal analysis in the frequency and time domains on the embedded DSP system. The frequency domain analysis is based on FFT (a computation-efficient algorithm for the DFT) and the time domain analysis is performed through identifying signal's time domain features. The main tasks are classified into two projects
 - (i). Project *F_Analysis*: Implementing FFT based on the Atmel DSPLib and using it to the following applications;
 - finding the frequency of a sine signal and showing the frequency in the Putty
 - calculating the spectrum of a rectangular signal and showing it in MATLAB; then figuring out how to determine the frequencies of the harmonics
 - (ii). Project *T_Analysis* (optional): making time analysis of signals in the following applications
 - finding the frequency of a sine signal and showing the frequency in the Putty
 - finding the time interval of two tone bursts generated using MATLAB and showing the result in the Putty
- To implement any DSP application, an efficient and effective approach is to find out the available DSP resource that is suited for the application. Atmel provides a good DSP library (*DSPLib*) [M13, M14] that includes a wide variety of DSP implementations, e.g., FFT, FIR and IIR filters, convolution. The most relevant functions are
 - ✓ `dsp16_trans_realcomplexfft` – compute complex FFT of a real signal
 - ✓ `dsp16_vect_complex_abs` – calculate the absolute value of a complex valued vector
 - ✓ `dsp16_vect_max` – find the maximum of a vector

Equipment, devices, hardware and software:

- a function generator, an oscilloscope, a multi-meter, a power supply
- the DIY circuit built during the first lab
- a computer installed with MATLAB, Atmel Studio 7, Flip and Putty; Use of your own laptop is highly recommended if you would like to work on the embedded systems at home as well
- A software framework (Lab-L3_DSP_framework.c) that includes ADC, DAC, and external trigger and USART

Setups:

- The setups for the projects in this lab session are similar to the one for Lab session 2 (referring to Lab-L2.pdf) except that no analog signals are needed to output from the DAC.

Contents

1. DSP Library	2
2. FFT	2
3. Implementation and tasks	3
References	4

1. DSP Library

The Atmel AVR32 DSP Library consists of a variety of digital signal processing functions. All the functions in the DSP Library are accessible from the AVR32 Software Framework. All the source code is available and fully compatible with GCC. The functions are optimized for the AT32UC family but also target-independent functions. The details on the library are available in [M13, M14].

All DSP types in the DSPLib include the notion of 16-bit and 32-bit fixed-point formats. It is important to understand this format in order to fastest debug an application.

The directory architecture of the DSPLib is shown below:

 AT32UC	AT32UC includes all the source code for the AT32UC targets and provided by the DSPLib.
 DOC	DOC includes the main documentations of the whole library.
 EXAMPLES	EXAMPLES regroups a lot of examples showing a way to use the functions.
 GENERIC	GENERIC regroups all the target-independent source code of the DSPLib.
 INCLUDE	INCLUDE includes all the common include files for the optimized and generic functions.
 UTILS	UTILS regroups useful tools and scripts for the DSP library.
 readme.html	readme.html is the entry point of the HTML documentation of the library. It gives accesses to doxygen documentations and many others useful data about functions, utilities, scripts, etc.

Each function in the basic library of the DSPLib follows this naming convention rule:

dspX_Y_Z(...)

- X is the type used (16 or 32)
- Y is category of the library (op, vect, trans, ...)
- Z is the short name of the function (cos, mul, ...)

AVR32 – Fixed point DSP Library Modules include two categories: *Basic* and *Advanced*:

- Basic
 - o Filtering
 - o Operators
 - o Signal generation
 - o Transforms
 - o Vectors
 - o Windowing
- Advanced
 - o IMA/DVI ADPCM
 - o Signal re-sampling

The full list of the functions is given in Appendix 1.

2. FFT

A fast Fourier transform (FFT) is an efficient algorithm to compute the discrete Fourier transform (DFT) and its inverse. There are many different FFT algorithms involving a wide range of mathematics, from simple complex-number arithmetic to group theory and number theory.

The FFT in the DSPLib is a radix 4 algorithm. In other words, the signal used in the FFT has a length of 4^n , e.g., 4, 16, 64, 256, 1024, 4096, ... The FFT functions are available in both 16 and 32 bits. But the 16-bit FFT is used here, i.e.,

✓ `dsp16_trans_realcomplexfft`

which computes complex FFT of a real signal.

3. Implementation and tasks

Implementation of signal analysis in the frequency domain and in the time domains is based on the software framework (Lab-L3_DSP_framework.c). In particular, four projects are suggested to be made in Atmel Studio 7: (1) *F_Analysis_1*, (2) *F_Analysis_2*, (3) *T_Analysis_1* and (4) *T_Analysis_2*, which are specified below. The services and drivers needed to include are:

- ABDAC – Audio Bitstream DAC (driver)
- ADC – Analog to Digital Converter (driver)
- DSPLib – Digital signal processing library (library)
- PDCA – Peripheral DMA Controller (driver)
- PM – Power Manager (driver)
- TC – Time/Counter (driver)
- USART – Universal Synchronous/Asynchronous Receiver/Transmitter (driver)
- System Clock Control (service)

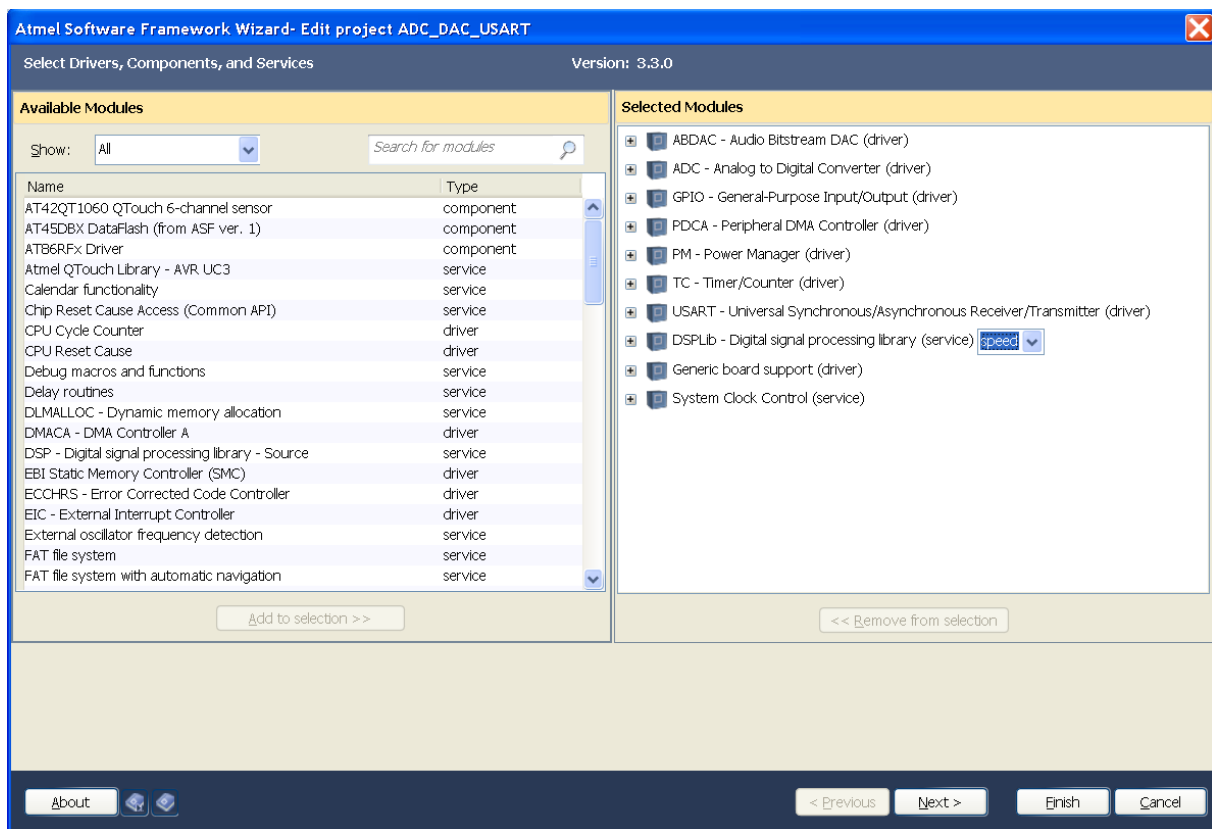


Fig. 1. The services and drivers (in the ASF) needed to include into the program/project.

The four projects are specified as follows

1. Project *F_Analysis*

- Implement FFT using the available FFT function `dsp16_trans_realcomplexfft`; in particular, search and find “DO DSP HERE” in main.c, and then you can see `DSP_REAL_BUFFER[ind]` which contains the input signal from ADC. Just below “DO DSP HERE” you add
`dsp16_trans_realcomplexfft(&DSP_COMP_BUFFER,&DSP_REAL_BUFFER,N_of_bits);`
in which `DSP_COMP_BUFFER` contains the DFT of the signal in `DSP_REAL_BUFFER`. To get real & imaginary parts, use `DSP_COMP_BUFFER[i]→real` and `DSP_COMP_BUFFER[i]→imag`
- apply it to the following applications for which you need to do a little programming, i.e., add a few lines suited for the applications
 - (1). *F_Analysis_1*: find the frequency of a sine signal and show the frequency in the Putty. In particular, one finds the magnitude spectrum (the absolute value of the spectrum), using `dsp16_vect_complex_abs(&DSP_REAL_BUFFER,&DSP_COMP_BUFFER,DATA_SIZE);` in

which DSP_REAL_BUFFER receives the magnitude spectrum; then one can use `dsp16_vect_max(&DSP_REAL_BUFFER, DATA_SIZE)`; to find the spectral peak, work a little more to find out the frequency of the peak; finally use USART to send the frequency out to Putty and show it.

- (2). F Analysis 2: calculate the spectrum of a rectangular signal and show (plot) it in MATLAB (the data is sent to MATLAB through USART); then determine and explain the frequencies in the spectrum

2. Project T Analysis (optional):

- Conducting time analysis of signals in the following applications
 - (1). T Analysis 1: find the frequency of a sine signal and show the frequency in the Putty. In this case, one needs to do a little programming, i.e., write a few command lines to determine the frequency according to the signal feature in the time domain
 - (2). T Analysis 2: Write a program (for the AVR UC3 evaluation board in the Atmel Studio 7) to find the time interval of two tone bursts which are generated using MATLAB. Specifically, use MATLAB to create two sine signals (with a sampling frequency of 44 100 Hz), each having a 1 V amplitude, a 4 kHz frequency and a duration of 4 cycles, and both separated by a time interval of 5 ms; then send it the evaluation board, determine the time interval using the program, and display in Putty.

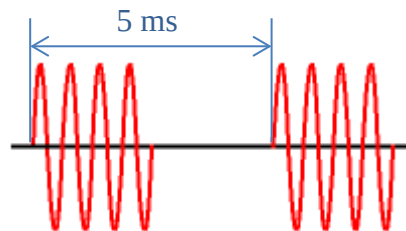


Fig. 2. Tone bursts generated using MATLAB and sent to the DSP system.

```
% Generate a signal of tone bursts
Fs = 44.1e3;      % one of the standard sampling frequencies for audio
signals
Tm = 10e-3;      % max duration in seconds
Nm = Fs*Tm;      % the max number of sampling points
t = (0:Nm)/Fs;   % t-axis
A0 = 1;          % Amplitude of the signal
f0 = 4e3;        % frequency of the signal in Hz
T0 = 1/f0;       % period of the signal in s
x1 = A0*sin(2*pi*f0*t).*(heaviside(t)-heaviside(t-4*T0));
% 1st tone burst
x2 = A0*sin(2*pi*f0*t).*(heaviside(t-5e-3)-heaviside(t-4*T0-5e-3));
% 2nd tone burst
x_t = x1+x2;

nn = 1:Nm; plot(t(nn),x_t(nn)); grid;

sound(x_t, Fs); %sends x_t with sample freq FS out to the speaker. Note: -
1.0 <= x_t <= 1.0.
```

References

- M6. Steven F. Barrett and Daniel J. Pack, *Atmel AVR Microcontroller Primer Programming and Interfacing*, 2nd Ed, Morgan & Claypool Publ., 2012.
<http://www.morganclaypool.com/doi/pdf/10.2200/S00427ED1V01Y201206DCS039>
- M7. * Steven F. Barrett and Daniel J. Pack, *Microcontrollers Fundamentals for Engineers and Scientists*, Morgan & Claypool Publ., 2006. (*For those who are new to Embedded Systems

- and have not taken any courses on Computer Systems or Embedded Systems).
- <http://www.morganclaypool.com/doi/pdf/10.2200/S00025ED1V01Y200605DCS001>
- M8. Atmel, *Atmel AVR32930: UC3-A3 Xplained Getting Started Guide*,
<http://www.atmel.com/Images/doc32163.pdf>
- M9. Atmel, *AVR32918: UC3-A3 Xplained Hardware User's Guide*,
<http://www.atmel.com/Images/doc32159.pdf>
- M10. Atmel 32-bit AVR[®] Microcontroller <http://www.atmel.com/Images/doc32072.pdf>
- M11. Atmel, *Atmel AVR4029: Atmel Software Framework - Getting Started*,
<http://www.atmel.com/Images/doc8431.pdf>
- M12. Atmel, *Atmel AVR4030: Atmel Software Framework - Reference Manual*,
<http://www.atmel.com/Images/doc8432.pdf>
- M13. Atmel, *AVR32718: AT32UC3 Series Software Framework DSPLib*
<http://www.atmel.com/Images/doc32076.pdf>
- M14. Atmel, *AVR32765: AVR32 DSPLib Reference Manual*
<http://www.atmel.com/Images/doc32120.pdf>
- M15. <http://www.atmel.com/>

Appendix 1. List of functions in DSP library [M14]

A.1 Filtering [Basic]

All the filtering functions currently supported by the DSP library.

- Finite Impulse Response Filter
- Infinite Impulse Response Filter
- Partial Infinite Impulse Response Filter
- Least Mean Square Filter
- Normalized Least Mean Square Filter
- Interpolation filter
- Lowpass FIR design

A.2 Operators [Basic]

All the fixed point operators functions currently implemented in the DSP library.

- Multiplication
- Division
- Sine
- Cosine
- Arc sine
- Arc cosine
- Absolute
- Square root
- Natural logarithm
- Binary logarithm or base 2 logarithm
- Common logarithm or base 10 logarithm
- Exponential
- Power
- Random

A.3 Signal generation [Basic]

All the signal generation functions currently supported by the DSP library.

- Periodic
 - Cosinusoidal
 - Sinusoidal
 - Square

- o Rectangular
- o Saw tooth
- o Dirac comb
- Non-periodic
 - o Noise
 - o Ramp
 - o Step
 - o Dirac

A.4 Transforms [Basic]

All the transforms functions currently supported by the DSP library.

- Complex Fast Fourier Transform
- Complex inverse Fast Fourier Transform
- Real to complex Fast Fourier Transform

A.5 Vectors [Basic]

All the vector management functions currently supported by the DSP library.

- Real
 - o Addition
 - o Addition with a real
 - o Subtraction
 - o Subtraction with a real
 - o Multiplication with a real
 - o Division with a real
 - o Dot multiplication
 - o Dot division
 - o Multiplication with an integer
 - o Division with an integer
 - o Power
 - o Minimum
 - o Maximum
 - o Negate
 - o Zero padding
 - o Copy
 - o Partial Convolution
 - o Convolution
- Complex
 - o Complex addition
 - o Complex subtraction
 - o Complex absolute
 - o Complex conjugate

A.6 Windowing [Basic]

All the windowing functions currently supported by the DSP library.

- Rectangular
- Bartlett
- Blackman
- Hamming
- Gauss
- Hann
- Kaiser

A.7 Advanced API

This group regroups all the functions available in the advanced library.

- ADPCM
- Re-Sampling